BÀI 8. CHUYÊN ĐỀ
THIẾT LẬP HỆ THỐNG ĐẢM BẢO CHẤT LƯỢNG DỰ ÁN PHẦN MỀM

MỤC LỤC

1. GIỚI THIỆU VỀ DỰ ÁN PHẦN MỀM 3
1.1. Tổng quan về dự án 3
1.2. Yêu cầu người dùng và phạm vi dự án 5
1.2.1. Yêu cầu người dùng 5
1.2.2. Phạm vi dự án 7
1.2.2.1. Phạm vi chức năng (Functional scope) 7
1.2.2.2. Phạm vi phi chức năng (Non-Functional scope) 7
1.2.2.3. Phạm vi không bao gồm (Out of scope) 8
1.3. Yêu cầu phần mềm 8
1.4. Đội dự án và đảm bảo chất lượng 9
2. THIẾT LẬP MỤC TIÊU VÀ YÊU CẦU CHẤT LƯỢNG CHO DỰ ÁN 10
2.1. Mục tiêu chất lượng 10
2.2. Yêu cầu chất lượng 11
3. XÁC ĐỊNH MÔ HÌNH PHÁT TRIỂN 12
4. THIẾT LẬP CÁC TIÊU CHUẨN, THỦ TỤC VÀ TÀI LIỆU QA 12
4.1. Tiêu chuẩn tài liệu 12
4.2. Thiết lập chuẩn mã nguồn (Coding Guidelines) 15
4.3. Thiết lập quy trình review 16
4.4. Thiết lập quy trình kiểm thử 16
4.5. Thiết lập danh mục kiểm tra 17
4.6. Thiết lập thủ tục QA 21
4.6.1. Thủ tục kiểm tra mã nguồn (Code review procedures) 21
4.6.2. Thủ tục kiểm thử phần mềm (Software testing procedures) 22
4.6.3. Thủ tục kiểm soát tài liệu (Document control procedures) 23
4.6.4. Thủ tục kiểm soát thay đổi (Change control procedures) 24

4.6.5. Thủ tục kiểm thử mã nguồn tự động (Automated code testing procedures)..... 25
4.6.6. Thủ tục báo cáo và phản hồi (Reporting and feedback procedures) ... 26
2.2.4.4. Thiết lập quy trình QA..... 27
2.2.4.5. Thiết lập danh mục kiểm tra 29
5. CÔNG CỤ HỖ TRỢ 30

BÀI 8. CHUYÊN ĐỀ
THIẾT LẬP HỆ THỐNG ĐẢM BẢO CHẤT LƯỢNG DỰ ÁN PHẦN MỀM

1. GIỚI THIỆU VỀ DỰ ÁN PHẦN MỀM

1.1. Tổng quan về dự án

HaUI Library là thư viện dành cho mọi độc giả có nhu cầu đọc sách. Có hai loại độc giả là: sinh viên và CB-Giảng viên. HaUI Library gồm có nhiều phòng được bố trí tại các điểm trường thuộc khu A, khu B và khu C. Dự kiến, HaUI Library muốn tin học hóa công việc quản lý thư viện bằng cách phát triển một phần mềm quản lý dựa trên nền tảng web. Ứng dụng cho phép chạy trên các trình duyệt phổ biến (Cốc cốc, Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera, Brave); chạy trên các thiết bị di động sử dụng hệ điều hành Adroid và IOS.

Hệ thống gồm 3 thành phần chính:

(1) Phần dành cho độc giả chạy trên nền web (hoặc ứng dụng di động):

- Độc giả là sinh viên và CBNV-Giảng viên được cấp thẻ độc giả. Mỗi độc giả được cấp 01 thẻ điện tử (hay còn gọi là thẻ mềm, thẻ chính là tài khoản để độc giả đăng nhập vào hệ thống HaUI Library), thẻ có thời hạn.

- Độc giả có thể tra cứu sách bằng cách tìm kiếm theo tựa đề hoặc tên tác giả. Hệ thống sẽ trả về danh sách các quyển sách phù hợp với yêu cầu tìm kiếm của người dùng bao gồm thông tin: tựa sách, tác giả, nhà xuất bản, năm xuất bản, lời giới thiệu, rating, số lượng sách còn ở các chi nhánh. Khi xem chi tiết, độc giả sẽ thấy thêm các lời bình về đầu sách của các độc giả khác.

- Dựa vào kết quả tìm kiếm, người dùng có thể đăng kí mượn sách bằng cách lựa chọn (click chọn) quyển sách cần mượn tương ứng với địa điểm muốn nhận sách từ thư viện. Hệ thống sẽ cho biết còn sách ở thư viện không, độc giả có quyền mượn sách không và mã số vị trí cuốn sách trong kho sách.

Lưu ý:

+ Nếu độc giả chưa đăng nhập vào hệ thống thì hệ thống tự đồng điều hướng (chuyển) sang phần đăng nhập (trang đăng nhập);

+ Mỗi độc giả, tại một thời điểm, chỉ được mượn tối đa 3 cuốn sách thuộc 3 đầu sách khác nhau (mỗi đầu sách chỉ được mượn 1 cuốn);

+ Sách trong kho được sắp xếp trên các giá sách, mỗi vị trí được đánh ký hiệu (gồm tên giá, tầng, ngăn) và ký hiệu này được đính kèm trên bìa cuốn sách.

- Trong quá trình mượn sách, độc giả được quyền bình luận, đánh giá mức độ (rating) cho quyển sách mà mình đang mượn. Chức năng này chỉ được hiển thị khi độc giả đã đăng nhập hệ thống.

- Ngoài ra, độc giả còn có thể đề xuất thư viện mua thêm sách, thông tin giới thiệu đầu sách gồm: tên đầu sách và tác giả. Chức năng này chỉ được hiển thị khi độc giả đã đăng nhập hệ thống.

(2) Phần dành cho thủ thư (quản thư hay chuyên viên tại phòng đọc) chạy trên nền web:

- Sau khi đăng kí mượn sách trên hệ thống, độc giả đến địa chỉ đã đăng ký, cung cấp mã số độc giả cho thủ thư để lấy sách mượn.

- Độc giả tự tìm sách trong kho theo mã (theo mã vị trí khi đăng ký mượn), mang sách tới quầy và làm thủ tục mượn sách với thủ thư. Khi này, thủ thư sẽ quét mã sách và tiến hành thủ tục cho độc giả mượn sách nếu độc giả đủ điều kiện và số lượng sách trong thư viện còn đủ số lượng cho mượn. Hệ thống tự kiểm tra điều kiện mượn sách của độc giả.

- Khi trả sách, độc giả phải trả sách đúng với địa chỉ thư viện đã mượn sách. Độc giả chỉ cần đưa các quyển sách cần trả, thủ thư sẽ quét mã sách để tiến hành thủ tục trả sách.

- Đối với độc giả là sinh viên sẽ bị giới hạn thời gian mượn 1 cuốn sách là 3 tháng; đối với độc giả là khách thì chỉ được mượn đọc tại thư viện; đối với độc

giả là CBNV và giảng viên thì không có giới hạn về thời gian nhưng chỉ được mượn tối đa 5 quyển trong một năm.

- Trường hợp trễ hạn, độc giả sẽ bị cấm mượn sách tương ứng với số ngày đã mượn trễ.

- Thủ thư thực hiện cấp thẻ điện tử, gia hạn và hủy thẻ độc giả.

-Thủ thư thực hiện nhập cuốn sách vào kho sách

- Thủ thư có thể thực hiện các thống kê và báo cáo gồm: (1) thống kê theo độc giả; (2) thống kê theo đầu sách; (3) thống kê theo lượt mượn – trả tương ứng với các đầu sách.

(3) Phần dành cho người quản lý chạy trên nền web:

- Người quản lý có thể thêm đầu sách mới và phân bổ lượng sách cho các điểm thư viện thuộc khu A, khu B, khu C sao cho hợp lý hơn.

- Người quản lý có thể thực hiện thống kê các yêu cầu mua sách cùng với số lượng của các yêu cầu đó.

- Người quản lý tạo tài khoản, cấp quyền và hủy tài khoản dành cho thủ thư.

Yêu cầu khác đối với hệ thống HaUI Library:

- Hệ thống đảm bảo tính dễ sử dụng cho người dùng;

- Giao diện web động tự động thay đổi cho phù hợp với người dùng trên các thiết bị di động.

1.2. Yêu cầu người dùng và phạm vi dự án

Phạm vi dự án là phần công việc cần thực hiện để hoàn thành và bàn giao hệ thống HaUI Library theo đúng yêu cầu người dùng.

1.2.1. Yêu cầu người dùng

Phần mềm HaUI Library sẽ cung cấp các chức năng sau cho từng nhóm người dùng: Độc giả, thủ thư, người quản lý thư viện

*** Đối với độc giả (sinh viên, CB-Giảng viên):**

- Đăng nhập hệ thống bằng tài khoản điện tử (thẻ mềm).
- Tìm kiếm sách theo tiêu đề hoặc tên tác giả.
- Xem chi tiết của sách: tựa đề, tác giả, nhà xuất bản, năm xuất bản, lời giới thiệu, số lượng còn lại ở các chi nhánh, lời bình và đánh giá từ độc giả khác.
- Đăng ký mượn sách, lựa chọn địa điểm nhận sách tại chi nhánh cụ thể (nếu còn sách và đủ điều kiện mượn).
- Biết được tình trạng còn sách và điều kiện mượn.
- Xem thông tin vị trí sách trong kho (vì kho mở, độc giả có thể tự tìm sách trong kho dựa trên mã vị trí).
- Đề xuất thư viện mua thêm sách (tên sách, tác giả), chỉ đề xuất khi đã đăng nhập.
- Đánh giá (rating) và bình luận sách đang mượn (chỉ khi đã đăng nhập).
- Theo dõi và tuân thủ các quy định về mượn và trả sách (ví dụ: thời gian mượn, địa điểm trả sách, số lượng giới hạn).

*** Đối với thủ thư:**

- Xác nhận việc mượn và trả sách bằng cách quét mã sách (quét mã sách để xác nhận mượn hoặc trả sách).
- Hệ thống tự động kiểm tra điều kiện mượn sách (đối tượng, số lượng, thời gian) và hiển thị thông tin cho thủ thư.
- Cấp, gia hạn hoặc hủy thẻ độc giả.
- Nhập sách mới vào kho, gán vị trí lưu trữ.
- Thống kê:
    + Theo độc giả.
    + Theo đầu sách.
    + Theo lượt mượn – trả.

*** Đối với người quản lý:**

- Thêm đầu sách mới.

- Phân bổ số lượng sách đến các chi nhánh (khu A, B, C).
- Thống kê và theo dõi các đề xuất mua sách từ độc giả.
- Quản lý tài khoản thủ thư: tạo, cấp quyền, hủy tài khoản.

*** Yêu cầu khác đối với hệ thống HaUI Library:**

- Hệ thống đảm bảo tính dễ sử dụng cho người dùng;
- Giao diện web động tự động thay đổi cho phù hợp với người dùng trên các thiết bị di động.

1.2.2. Phạm vi dự án

1.2.2.1. Phạm vi chức năng (Functional scope)

- Phát triển hệ thống web và ứng dụng di động cho các nhóm người dùng: độc giả, thủ thư, quản lý.
- Xây dựng các chức năng chính:
  + Tra cứu sách, xem thông tin chi tiết.
  + Đăng ký mượn sách, đánh giá và bình luận.
  + Đề xuất sách mới.
  + Cấp/Quản lý thẻ độc giả.
  + Thực hiện mượn – trả sách.
  + Thống kê và báo cáo sử dụng.
  + Quản lý tài khoản và phân quyền.
  + Nhập sách và phân bổ sách (đầu sách) đến các chi nhánh thư viện.

1.2.2.2. Phạm vi phi chức năng (Non-Functional scope)

- Tính khả dụng: Hệ thống hoạt động ổn định 24/7, phục vụ đồng thời tối thiểu 1.000 người dùng.
- Hiệu năng:
  + Truy vấn tìm kiếm sách có phản hồi dưới 3 giây.
  + Dữ liệu được sao lưu định kỳ hàng ngày.
- Tính bảo mật:

    - Chỉ người dùng hợp lệ mới truy cập được chức năng.
    - Phân quyền chặt chẽ theo vai trò (độc giả, thủ thư, quản lý).
- Tính tương thích:
    - Ứng dụng web và mobile (đa nền tảng: desktop, Android, iOS).
    - Hệ thống hỗ trợ trên trình duyệt phổ biến (Chrome, Firefox, Edge, Safari, Opera, Brave).
    - Hỗ trợ các hệ điều hành Android và iOS.
    - Responsive UI (giao diện thích ứng).
- Giao diện và trải nghiệm người dùng:
    - Giao diện web responsive, tự điều chỉnh phù hợp thiết bị.
    - Dễ sử dụng, hỗ trợ điều hướng logic và rõ ràng cho từng loại người dùng.
- Hệ thống có cơ sở dữ liệu quản lý độc giả, sách, lượt mượn – trả, đề xuất sách, tài khoản người dùng,…

1.2.2.3. Phạm vi không bao gồm (Out of scope)

- Thanh toán hoặc xử lý phí trễ hạn (nếu có phát sinh sau này).
- Tích hợp RFID/máy mượn sách tự động.
- Tích hợp thư viện số (eBook, tài liệu điện tử).
- Gửi email/SMS tự động nhắc trả sách.

1.3. Yêu cầu phần mềm

Yêu cầu nghiệm vụ (Business Requirements - BR) trên Bảng 1, yêu cầu hệ thống (System Requirements - SR) trên Bảng 2, yêu cầu phi chức năng (Non-Functional Requirements - NFR) trên Bảng 3.

Bảng 1. Yêu cầu nghiệp vụ

Mã yêu cầu	Nội dung yêu cầu
BR1	Người dùng chỉ truy cập chức năng sau khi đăng nhập hợp lệ
BR2	Mỗi độc giả có một thẻ điện tử riêng có thời hạn

BR3	Chỉ độc giả đăng nhập mới được mượn sách, bình luận, đề xuất mua sách
BR4	Độc giả được tìm kiếm sách theo tiêu đề, tác giả
BR5	Cho phép đăng ký mượn sách theo chi nhánh
BR6	Quy định mượn sách theo từng đối tượng độc giả
BR7	Cho phép đánh giá và bình luận sách đã mượn
BR8	Cho phép đề xuất mua sách mới với tên và tác giả
BR9	Thủ thư xác nhận mượn/trả sách bằng quét mã sách
BR10	Quản lý trễ hạn: cấm mượn tương ứng số ngày vi phạm
BR11	Thủ thư cấp, gia hạn, hủy thẻ độc giả
BR12	Nhập sách mới vào kho và gán vị trí lưu trữ
BR13	Xem thống kê theo độc giả, đầu sách, lượt mượn-trả
BR14	Quản lý thêm đầu sách và phân bổ số lượng đến chi nhánh
BR15	Thống kê yêu cầu mua sách từ độc giả
BR16	Quản lý tài khoản thủ thư: tạo, cấp quyền, hủy tài khoản

Bảng 2. Yêu cầu hệ thống

Mã yêu cầu	Nội dung yêu cầu
SR1	Hiển thị kết quả tìm kiếm: tựa sách, tác giả, NXB, năm XB, rating, chi nhánh còn sách
SR2	Kiểm tra điều kiện mượn sách: còn sách không, có vi phạm quy định không
SR3	Tự động xác định trạng thái mượn hợp lệ khi thủ thư quét mã sách

Bảng 3. Yêu cầu phi chức năng

Mã yêu cầu	Nội dung yêu cầu
NFR1	Hệ thống xử lý tối thiểu 1.000 truy cập đồng thời
NFR2	Thời gian phản hồi tìm kiếm dưới 3 giây
NFR3	Dữ liệu được sao lưu định kỳ hàng ngày
NFR4	Giao diện thân thiện, dễ dùng, hiển thị rõ ràng trên mọi loại thiết bị
NFR5	Hệ thống hỗ trợ web/mobile, chạy trên Chrome, Firefox, Edge, Safari, Opera, Brave
NFR6	Giao diện responsive phù hợp với thiết bị di động

1.4. Đội dự án và đảm bảo chất lượng

Bảng 4. Thành viên đội dự án

TT	Vai trò	Họ tên	Ghi chú

1	Quản lý dự án (PM)	Trần Minh G	Phụ trách điều phối, quản lý tiến độ, phê duyệt tài nguyên
2	QA leader	Nguyễn Văn A	Lập kế hoạch QA/QC/QE, tổng hợp kết quả chất lượng
3	SQA team	Trần Thị B: Reviewer Trần Văn C: Reviewer Nguyễn Thị D: Tester	Review yêu cầu, kiểm thử hệ thống, đánh giá chất lượng
4	Developer	Hoàng Văn E, Lê Thị F	Coding, viết unit test, fix bug, hỗ trợ kiểm thử
5	Trưởng nhóm thiết kế (kiêm nhiệm)	Lê Thị F	Đảm bảo mô hình thiết kế đúng kiến trúc và yêu cầu
6	Khách hàng nội bộ	Nguyễn Quốc H (Đại diện người dùng)	UAT, đánh giá tính dễ dùng, xác nhận nghiệm thu

2. THIẾT LẬP MỤC TIÊU VÀ YÊU CẦU CHẤT LƯỢNG CHO DỰ ÁN

2.1. Mục tiêu chất lượng

Dự án HaUI Library hướng tới việc phát triển một hệ thống phần mềm có chất lượng cao, đảm bảo các đặc tính như hiệu năng, dễ sử dụng, an toàn và bảo trì tốt. Để đạt được điều này, các mục tiêu chất lượng sẽ được xác định và kiểm soát dựa trên mô hình đặc tính chất lượng phần mềm ISO/IEC 25010:2011, IEEE 730, ISO/IEC 12207, và CMMI-DEV v2.0.

Các mục tiêu chất lượng của dự án:

– Tính năng đầy đủ và chính xác: Cung cấp đầy đủ các chức năng tra cứu, mượn – trả sách, thống kê, quản lý thẻ, đề xuất sách, phân quyền người dùng.

– Tính khả dụng cao: Hệ thống cần hoạt động ổn định trên các trình duyệt và hệ điều hành phổ biến, đảm bảo tính sẵn sàng phục vụ liên tục.

– Thân thiện người dùng: Giao diện dễ sử dụng cho cả sinh viên, cán bộ – giảng viên, thủ thư và người quản lý.

– Hiệu suất xử lý tốt: Hệ thống cần phản hồi nhanh các thao tác tra cứu, mượn trả sách, truy cập dữ liệu.

– Tính bảo mật: Đảm bảo an toàn thông tin người dùng và dữ liệu thư viện.

– Tính tương thích: Hệ thống phải chạy ổn định trên các thiết bị (PC, laptop, tablet, điện thoại) và trình duyệt (Chrome, Cốc Cốc, Edge…).

– Khả năng bảo trì và mở rộng: Cho phép cập nhật thông tin đầu sách, tính năng mới, hỗ trợ nhiều điểm thư viện.

2.2. Yêu cầu chất lượng

Yêu cầu chất lượng được xây dựng dựa theo 8 đặc tính chất lượng trong ISO/IEC 25010:2011 (Bảng 5)

Bảng 5. Yêu cầu chất lượng được xác định theo ISO/IEC 25010:2011

Đặc tính chính	**Yêu cầu chất lượng cụ thể cho HaUI Library**
1. Functional suitability (Tính phù hợp chức năng)	- Đảm bảo hệ thống thực hiện đúng yêu cầu nghiệp vụ. - Đáp ứng đúng các chức năng nghiệp vụ: đăng nhập, tìm kiếm, mượn-trả, quản lý thẻ, đề xuất sách. - Kiểm soát đúng giới hạn mượn và thời hạn theo loại độc giả. - Hiển thị đầy đủ thông tin sách, tình trạng kho, đánh giá, đề xuất.
2. Performance efficiency (Hiệu năng xử lý)	- Hệ thống phản hồi trong vòng < 3 giây cho các thao tác tìm kiếm, tra cứu. - Tối ưu hóa truy vấn dữ liệu cho khối lượng người dùng đồng thời. - Khả năng chịu tải: Hỗ trợ ≥ 2.000 user truy cập cùng lúc. - Ứng dụng web và mobile nhẹ, tải nhanh, hỗ trợ mạng chậm.
3. Compatibility (Khả năng tương thích)	- Tương thích với các trình duyệt: Chrome, Cốc Cốc, Edge. - Giao diện responsive, hiển thị đúng trên thiết bị Android, iOS.
4. Usability (Tính dễ sử dụng)	- Giao diện đơn giản, trực quan, ngôn ngữ dễ hiểu. - Cung cấp hỗ trợ sử dụng (tooltip, thông báo lỗi rõ ràng).

	- Có khả năng truy cập nhanh các chức năng chính chỉ sau 2–3 thao tác.
5. Reliability (Tính tin cậy)	- Hệ thống duy trì hoạt động ổn định liên tục 24/7. - Không mất dữ liệu khi xảy ra lỗi. - Cơ chế sao lưu dữ liệu định kỳ.
6. Security (Tính bảo mật)	- Mã hóa dữ liệu đăng nhập. - Phân quyền chặt chẽ (độc giả, thủ thư, quản lý). - Bảo vệ khỏi truy cập trái phép vào chức năng quản lý.
7. Maintainability (Khả năng bảo trì)	- Kiến trúc phân lớp rõ ràng, dễ nâng cấp. - Tài liệu thiết kế rõ ràng, dễ bảo trì hệ thống.
8. Portability (Tính di động)	- Có thể triển khai trên nhiều nền tảng: web, mobile. - Hệ thống dễ dàng triển khai ở các chi nhánh thư viện khác nhau.

3. XÁC ĐỊNH MÔ HÌNH PHÁT TRIỂN

Nhóm dự án vận dụng mô hình quy trình Waterfall cho xây dựng quy trình phát triển phần mềm. Dự án phát triển phần mềm HaUI Library có các yêu cầu ổn định, rõ ràng, minh bạch.

4. THIẾT LẬP CÁC TIÊU CHUẨN, THỦ TỤC VÀ TÀI LIỆU QA

4.1. Tiêu chuẩn tài liệu

Các tiêu chuẩn cho tài liệu QA của dự án phần mềm HaUI Library:

(1) Kế hoạch đảm bảo chất lượng (QA_Plan.docx):

– Mục tiêu

– Tiêu chuẩn áp dụng:

+ ISO/IEC 25010 (đặc tính chất lượng), IEEE 730 (SQA Plan), IEEE 829 (test doc).

+ Quy trình kiểm thử:

- Unit test: cho hàm xử lý mượn/trả

- Integration test: mượn sách kết hợp kiểm tra số lượng kho,…
- System test: toàn bộ quy trình mượn – trả,…
- Acceptance test

+ Checklist kiểm tra code

+ Kế hoạch review: review tài liệu SRS, SDD, Test Cases.

(2) Quy trình phát triển (Dev_Workflow_Design.docx): Vận dụng mô hình Waterfall, quy trình phát triển phần mềm tuân theo trình tự tuyến tính với các giai đoạn kế tiếp nhau. Mỗi giai đoạn chỉ bắt đầu khi giai đoạn trước đã hoàn tất và được nghiệm thu:

– Luồng công việc (Hình 1) và mô tả quy trình làm việc theo từng giai đoạn (Bảng 2).



Hình 1. Sơ đồ luồng công việc chính trong quy trình

Bảng 6. Mô tả quy trình làm việc theo từng giai đoạn trong quy trình

Giai đoạn	Hoạt động chính	Tài liệu đầu ra	Người chịu trách nhiệm
1. Phân tích yêu cầu	Thu thập, phân tích nghiệp vụ thư viện, xác định chức năng cần thiết	Ghi chú yêu cầu, Use case draft	A (BA)
2. Đặc tả yêu cầu phần mềm	Viết tài liệu SRS: yêu cầu chức năng & phi chức năng, mockup sơ bộ	`SRS.docx`, Function_Specs	A phối hợp B (Tester)
3. Thiết kế hệ thống	Thiết kế kiến trúc tổng thể, cơ sở dữ liệu, luồng xử lý, giao diện	`SDD.docx`, Function_Designs	C (System architect)
4. Lập trình (Coding)	Phát triển mã nguồn theo thiết kế đã duyệt. Tuân thủ coding standards	Mã nguồn chức năng	D (Developer) phối hợp E
5. Kiểm thử	Viết test case, thực hiện test, ghi nhận lỗi	`Test_Cases.docx`, `Defect_Report.docx`, `Test_Summary_Report.docx`	B (Tester)
6. Bàn giao & bảo trì	Tổng hợp kết quả, demo cho quản lý thư viện, sửa lỗi sau kiểm thử	`Retrospective.docx`, Log lỗi đã sửa	Tất cả, điều phối bởi A

– Phân vai nhiệm vụ thành viên nhóm dự án trên Bảng 3.

Bảng 7. Phân công vai trò thành viên đội dự án

Thành viên	Vai trò chính	Vai trò phụ
A	BA (Business Analyst)	Quản lý tài liệu, nghiệm thu yêu cầu
B	Tester / QA	Hỗ trợ viết yêu cầu, kiểm thử hệ thống
C	System architect	Review yêu cầu, thiết kế giải pháp
D	Developer chính	Viết mã, fix lỗi, hỗ trợ kiểm thử
E	Developer phụ	Code bổ trợ, hỗ trợ C & D, refactor

(3) Chuẩn bố cục và trình bày các tài liệu (Doc_Stand_Guide.docx):

– Font: Times New Roman, size 13.

– Đánh số mục: 1, 1.1, 1.1.1...

– Tên file: TênTàiLiệu_Version.docx (ví dụ: SRS_v1.1.docx).

– Mỗi chức năng có file riêng: Login_Spec.docx, SearchBook_Design.docx.

(4) Tài liệu đặc tả yêu cầu phần mềm. Đặc tả các yêu cầu chức năng và phi chức năng của hệ thống, làm cơ sở để phát triển, kiểm thử và đánh giá phần mềm. Tài liệu: SRS.docx (Software Requirements Specification)

(5) Tài liệu thiết kế phần mềm: Mô tả kiến trúc phần mềm, thiết kế hệ thống và các module thành phần. Nội dung bao gồm sơ đồ kiến trúc, cơ sở dữ liệu, luồng xử lý chính. Tài liệu: SDD.docx (Software Design Document)

(6) Kế hoạch kiểm thử: Mô tả chiến lược kiểm thử tổng thể: phạm vi, phương pháp, công cụ, nguồn lực, lịch trình và loại kiểm thử (unit test, integration test, system test). Tài liệu: Test_Plan.docx

(7) Danh sách test case**:** Bao gồm danh sách các trường hợp kiểm thử cụ thể được viết dựa trên yêu cầu phần mềm. Mỗi test case gồm ID, bước thực hiện, đầu vào, kết quả mong đợi và trạng thái. Tài liệu: Test_Cases.docx

(8) Checklist review mã nguồn: bao gồm tiêu chí dùng để kiểm tra mã nguồn: (-) Tên biến theo camelCase; (-) Mỗi hàm tối đa 25 dòng; (-) Có comment

cho các function chính; (-) Không để code thừa hoặc bị lặp. Tài liệu: Code_Review_Checklist.docx

(9) Báo cáo lỗi: Ghi lại chi tiết các lỗi được phát hiện trong quá trình kiểm thử: mô tả lỗi, mức độ nghiêm trọng, người phát hiện, người xử lý, trạng thái xử lý. Tài liệu: Defect_Report.docx.

(10) Báo cáo tổng kết kiểm thử: Ghi lại số lượng test case pass/fail, thống kê lỗi, tổng hợp kết quả kiểm thử. Tài liệu: Test_Summary_Report.docx.

(11) Báo cáo retrospective (nếu có): Tài liệu tổng kết sau mỗi sprint: những gì đã làm tốt, chưa tốt và bài học kinh nghiệm cho các sprint tiếp theo (nếu có). Tài liệu: Retrospective.Docx.

4.2. Thiết lập chuẩn mã nguồn (Coding Guidelines)

(1) Quy tắc chung:

- Ngôn ngữ lập trình: JavaScript (Frontend) + Python/Java (Backend).

- Phiên bản: Tuân thủ LTS (Node.js 18+, Python 3.10+).

- IDE/Editor: VS Code (cài extension ESLint, Prettier).

(2) Quy tắc đặt tên:

- Tên biến (Variables): dùng camelCase, rõ nghĩa.

- Tên hàm (Functions): dùng camelCase, biểu thị hành động (ví dụ: calculateScore).

- Tên lớp (Classes): dùng PascalCase.

- Hằng số (Constants): viết hoa toàn bộ, dùng gạch dưới (ví dụ: MAX_VALUE).

(3) Quy tắc mã nguồn:

- Cấu trúc code:

+ Mỗi file chỉ chứa 1 class/component (trừ utils/helpers);

+ Độ dài hàm ≤ 30 dòng, độ phức tạp cyclomatic ≤ 15.

+ Comment rõ ràng

- Bắt buộc dùng try-catch cho các thao tác I/O (DB, API).
- Tài liệu tham khảo:

+ Google JavaScript Style Guide

+ PEP 8 – Python Style Guide

+ Git Best Practices

4.3. Thiết lập quy trình review

Xác định rõ trình tự thực hiện hoạt động review trong vòng đời phát triển phần mềm, đảm bảo chất lượng và phát hiện lỗi sớm.

Quy trình review được áp dụng cho hợp đồng, tài liệu yêu cầu (SRS), thiết kế (SDD), mã nguồn. Các bước thực hiện:

(1) Chuẩn bị: chọn đối tượng review (SRS, thiết kế, code...), phân công người review

(2) Phân phối tài liệu: gửi tài liệu cho các reviewer cùng với checklist kiểm tra

(3) Thực hiện review cá nhân: mỗi reviewer rà soát và ghi chú phát hiện

(4) Họp tổng hợp (optional): thảo luận và thống nhất lỗi/cải tiến

(5) Ghi nhận kết quả: lập biên bản review và phân loại mức độ ảnh hưởng

(6) Theo dõi khắc phục: tác giả cập nhật, reviewer xác nhận sửa lỗi

Lưu ý:

- Áp dụng peer review cho tài liệu, và code review với >30% mã thay đổi.
- Reviewer phải độc lập với người viết tài liệu hoặc người viết mã.

4.4. Thiết lập quy trình kiểm thử

Quy trình kiểm thử xác định rõ trình tự thực hiện hoạt động kiểm thử trong vòng đời phát triển phần mềm, đảm bảo chất lượng và phát hiện lỗi sớm. Quy trình kiểm thử phần mềm được áp dụng cho các chức năng phần mềm ở tất cả các cấp độ kiểm thử.

Các bước thực hiện:

(1) Xác định phạm vi kiểm thử: dựa trên SRS, kế hoạch test

(2) Thiết kế test case & test data: tạo test case, dữ liệu tương ứng

(3) Thiết lập môi trường kiểm thử: cấu hình hệ thống, dữ liệu demo

(4) Thực thi kiểm thử: chạy test case và ghi lại kết quả

(5) Ghi nhận lỗi: báo lỗi (nếu có), phân loại theo mức độ nghiêm trọng

(6) Kiểm thử hồi quy (re-test): sau khi lỗi được sửa

(7) Tổng hợp và báo cáo: lập báo cáo kiểm thử (Test Report)

Các cấp độ kiểm thử được áp dụng:

- Unit test: do developer thực hiện.

- Integration test: kiểm tra sự kết hợp giữa các module.

- System test: kiểm tra toàn bộ hệ thống theo luồng nghiệp vụ.

- Acceptance test (UAT): tester kiểm thử (kiểm thử alpha), và sau đó khách hàng xác nhận tính đúng đắn trước khi triển khai (kiểm thử beta).

4.5. Thiết lập danh mục kiểm tra

1. Checklist review tài liệu yêu cầu (SRS)

Bảng 8. Checklist review tài liệu yêu cầu

TT	Nội dung kiểm tra	Tiêu chí đạt	Kết quả (v/x)	Ghi chú
1	Đầy đủ chức năng nghiệp vụ			
2	Yêu cầu phi chức năng			
3	Thuật ngữ thống nhất			

2. Checklist thiết kế DATABASE

Bảng 9. Checklist thiết kế CSDL

TT	Bảng cần kiểm tra	Trường bắt buộc	Ràng buộc	Kết quả	Ghi chú
1					
2					

3. Checklist coding standard (BACKEND - JAVA)

Bảng 10. Checklist coding

TT	Hạng mục	Quy tắc	Ví dụ đạt chuẩn	Kết quả
1	Đặt tên biến	camelCase, không dùng tiếng Việt	studentCode thay vì maSV	
2	Xử lý ngoại lệ	Bắt SQLException khi truy vấn database	try-catch rõ ràng	

4. Checklist test case chức năng "đăng ký học phần"

Bảng 11. Checklist test case

TT	Test case	Đầu vào	Kết quả mong đợi	Thực tế
1				
2				

5. Checklist UI/UX giao diện sinh viên

Bảng 12. Checklist UI/UX

TT	Thành phần	Tiêu chí	Ảnh minh họa	Đạt/chưa
1				
2				

6. Checklist bảo mật (áp dụng cho QE)

Bảng 13. Checklist QE

TT	Hạng mục	Phương pháp kiểm tra	Công cụ	Kết quả
1				
2				

7. Checklist triển khai

Bảng 14. Checklist triển khai

TT	Công việc	Yêu cầu	Người phụ trách	Hoàn thành
1	Backup database	Sao lưu toàn bộ dữ liệu production	Nguyễn Văn A	
2	Kiểm tra server	CPU ≤ 80%, ram ≥ 4Gb	Phạm Thị D	

8. Checklist đánh giá chất lượng cuối dự án

Bảng 15. Checklist đánh giá cuối dự án

TT	Chỉ số	Mục tiêu	Thực tế	Đánh giá
1	Tỷ lệ test case pass	≥ 95%		
2	Số lỗi critical	0		

9. QA test checklist

Bảng 16. QA testing checklist

TT	Test checklist	Đã kiểm tra	Ghi chú
1	User story có tiêu chí chấp nhận rõ ràng (Acceptance criteria)		
2	Có unit test cho tất cả các hàm nghiệp vụ chính		
3	Tỷ lệ bao phủ kiểm thử (Test coverage) đạt ≥ 80%		
4	Chạy thành công các kiểm thử tự động (CI/CD)		
5	Không có bug nghiêm trọng (Severity 1 hoặc 2) tồn tại sau Sprint		
6	Kết quả kiểm thử được ghi nhận trong hệ thống quản lý test (nếu có)		
7	Kiểm thử hồi quy được thực hiện sau mỗi lần cập nhật code chính		
8	Đáp ứng yêu cầu hiệu năng: < 2s cho truy vấn điểm		
9	Trình bày UI/UX đúng bản thiết kế		

10. Code review checklist

Bảng 17. Code review checklist

TT	Tiêu chí	Mức độ ưu tiên	Ghi chú
1	Code dễ đọc, có đặt tên biến/hàm rõ ràng	Cao	
2	Không lặp mã (code duplication)	Trung bình	
3	Comment rõ ràng nếu đoạn code phức tạp	Cao	
4	Tuân thủ quy ước code chuẩn (coding convention)	Cao	
5	Không commit mã test/thử nghiệm vào nhánh chính	Cao	
6	Xử lý ngoại lệ rõ ràng (error handling)	Trung bình	
7	Có test tương ứng nếu thêm chức năng mới	Cao	

8	Không để lại mã thừa, mã bị comment bỏ	Trung bình	

Waterfall là một trong những mô hình thuộc nhóm phương pháp Agile được áp dụng rộng rãi trong phát triển phần mềm nhờ tính linh hoạt, minh bạch và khả năng thích ứng cao. Scrum cho phép thay đổi yêu cầu/chức năng, linh hoạt, dễ thích nghi.

Nhóm dự án cần thống nhất mô hình quy trình phát triển phần mềm để xây dựng kế hoạch phát triển. Mô hình quy trình giúp quản lý tốt hơn trong quản lý phát triển dự án, cung cấp cấu trúc rõ ràng cho các hoạt động như lập kế hoạch, thiết kế, kiểm thử, triển khai,... giúp dễ quản lý tiến độ và chất lượng. Hỗ trợ kiểm soát chất lượng (QA-QC-QE) trong toàn bộ dự án.

Một số tiêu lựa chọn mô hình phát triển phần mềm:

- Phù hợp với đặc thù dự án: Mỗi dự án có yêu cầu, độ phức tạp, nguồn lực, và thời gian khác nhau. Mô hình phù hợp giúp tận dụng tối đa nguồn lực và kiểm soát rủi ro.

- Lựa chọn dựa trên mức độ rõ ràng của yêu cầu ban đầu:

  + Nếu yêu cầu ổn định, rõ ràng → có thể chọn Waterfall. Ví dụ với phần mềm quản lý điểm truyền thống.

  + Nếu yêu cầu thay đổi linh hoạt → chọn Agile/Scrum. Ví dụ với ứng dụng giáo dục có AI đề xuất học tập.

- Lựa chọn dựa trên độ lớn và độ phức tạp của dự án: Dự án lớn, phức tạp → ưu tiên Spiral hoặc Incremental để dễ kiểm soát rủi ro. Tuy nhiên, nếu đội thực hiện dự án ít người, ưu tiên chọn Incremental.

- Lựa chọn dựa trên thời gian thực hiện và ngân sách cho dự án: Với dự án gấp rút (thời gian ngắn), cần sản phẩm nhanh → Rapid Application Development (RAD) hoặc Agile. Cần lưu ý, RAD/Agile nhanh nhưng dễ phát sinh chi phí do thay đổi liên tục.

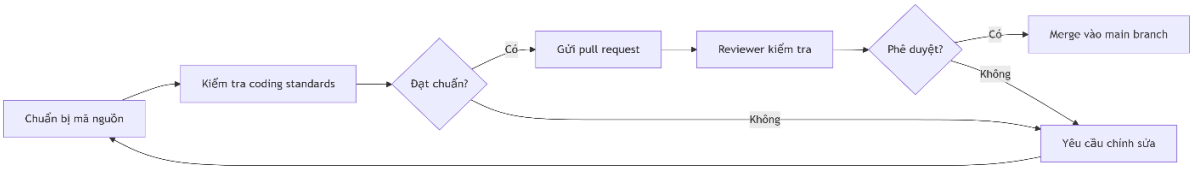
- Lựa chọn dựa trên khả năng tương tác với khách hàng: Nếu khách hàng sẵn sàng tham gia thường xuyên → chọn mô hình Agile/Scrum để linh hoạt cập nhật yêu cầu.

- Lựa chọn dựa trên mức độ kiểm soát và tiêu chuẩn hóa quy trình: Dự án thuộc tổ chức đòi hỏi quy trình kiểm soát nghiêm ngặt (ví dụ: phần mềm y tế, giáo dục) → V-model hoặc Spiral sẽ phù hợp. Cần lưu ý, bổ sung thêm tiêu chuẩn kèm theo trong các dự án đặc thù có tiêu chuẩn riêng (Dự án y tế: V-model + ISO 13485 (tiêu chuẩn về thiết bị y tế))

4.6. Thiết lập thủ tục QA

4.6.1. Thủ tục kiểm tra mã nguồn (Code review procedures)

- Các bước thực hiện (minh họa trên Hình 2):
  + Trước khi mã nguồn được tích hợp vào kho mã chính (main branch), tất cả các thay đổi phải được kiểm tra qua quy trình review.
  + Mã nguồn phải tuân thủ coding standards đã được thiết lập (ví dụ: quy tắc đặt tên biến, cấu trúc file, comment).
  + Một nhóm reviewer (thành viên phát triển khác hoặc leader) sẽ kiểm tra mã nguồn và ghi lại các nhận xét hoặc yêu cầu sửa đổi.
  + Nếu mã nguồn không đáp ứng yêu cầu, developer sẽ chỉnh sửa lại và tiếp tục gửi mã nguồn cho review lại.



Hình 2. Sơ đồ luồng thủ tục kiểm tra mã nguồn

- Công cụ hỗ trợ:
  + SonarQube (để tự động kiểm tra mã nguồn và tìm ra các lỗi tiềm ẩn);
  + GitHub pull requests (để theo dõi quá trình review).
- Vai trò của thành viên nhóm dự án:

+ Developer (Lập trình viên): Cung cấp mã nguồn cần được kiểm tra, chỉnh sửa lại mã nguồn khi có nhận xét hoặc yêu cầu sửa đổi.

+ Reviewer (Người kiểm tra mã): Thành viên phát triển khác hoặc leader sẽ thực hiện việc kiểm tra mã nguồn, cung cấp nhận xét, đánh giá mã nguồn và yêu cầu thay đổi nếu cần.

+ Team lead (Trưởng nhóm): Quản lý quy trình kiểm tra mã nguồn, đảm bảo rằng tất cả mã nguồn đều được kiểm tra và đáp ứng tiêu chuẩn chất lượng.

+ Tools Admin (Quản trị viên công cụ): Cài đặt và cấu hình các công cụ tự động như SonarQube để hỗ trợ kiểm tra mã nguồn tự động.

4.6.2. Thủ tục kiểm thử phần mềm (Software testing procedures)

- Các bước thực hiện (Hình 3):

+ Chuẩn bị kế hoạch kiểm thử (Test plan): Xác định phạm vi kiểm thử, các loại kiểm thử (unit, integration, system, regression), công cụ kiểm thử, lịch trình và nguồn lực.

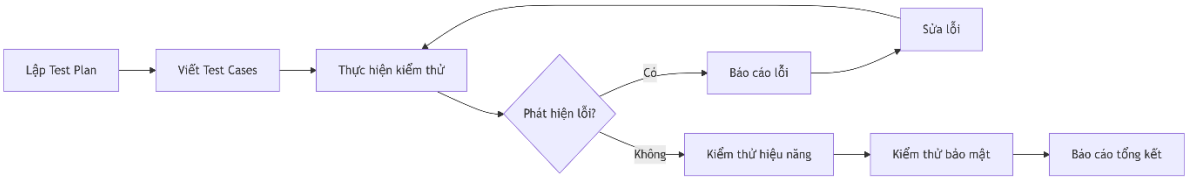
+ Xây dựng test cases: Dựa trên tài liệu đặc tả yêu cầu phần mềm (SRS), tạo ra danh sách các test case chi tiết, bao gồm các bước kiểm tra, dữ liệu đầu vào, kết quả mong đợi và điều kiện tiên quyết.

+ Thực hiện kiểm thử: Tiến hành kiểm thử theo từng giai đoạn (unit testing, integration testing, system testing) và theo dõi kết quả.

+ Báo cáo lỗi: Mọi lỗi phát hiện trong quá trình kiểm thử sẽ được ghi lại trong Defect Report, phân loại mức độ lỗi và đưa ra kế hoạch khắc phục.

+ Kiểm thử hiệu suất: Đo kiểm thử khả năng chịu tải và hiệu suất phần mềm dưới môi trường người dùng thực tế (sử dụng công cụ như JMeter).

+ Kiểm thử bảo mật: Đảm bảo phần mềm không có lỗ hổng bảo mật, sử dụng các công cụ như OWASP ZAP.



Hình 3. Sơ đồ luồng công việc thủ tục kiểm thử phần mềm

- Vai trò của thành viên nhóm dự án:

+ Test manager (Quản lý kiểm thử): Chịu trách nhiệm lên kế hoạch kiểm thử tổng thể, xác định các mục tiêu và phạm vi kiểm thử.

+ Test designer (Người thiết kế kiểm thử): Xây dựng các test case chi tiết dựa trên yêu cầu phần mềm (SRS).

+ Tester (Người kiểm thử): Thực hiện kiểm thử theo các test case đã xây dựng, báo cáo kết quả kiểm thử, phát hiện và ghi lại các lỗi.

+ Performance tester (Kiểm thử hiệu suất): Thực hiện kiểm thử hiệu suất và đánh giá khả năng chịu tải của phần mềm.

+ Security tester (Kiểm thử bảo mật): Chịu trách nhiệm thực hiện các kiểm thử bảo mật, đảm bảo phần mềm không có lỗ hổng.

4.6.3. Thủ tục kiểm soát tài liệu (Document control procedures)

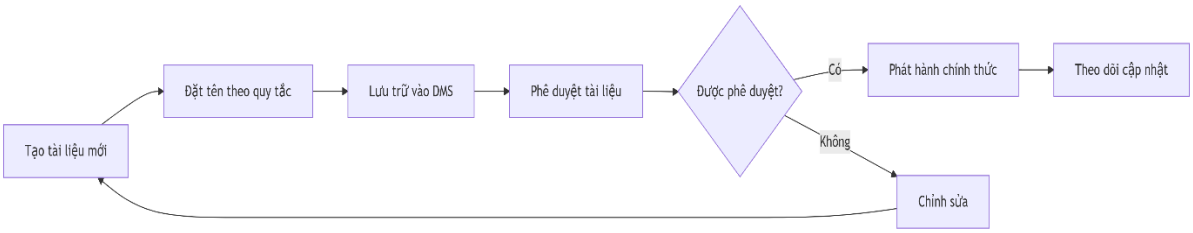
- Các bước thực hiện (Hình 4):

+ Quy định quản lý tài liệu: Mỗi tài liệu cần có một quy trình quản lý phiên bản rõ ràng, đặt tên file theo quy tắc nhất quán (ví dụ: ChucNang_Specs.docx, Login_Design.docx).

+ Quản lý tài liệu qua hệ thống quản lý (Document Management System): Lưu trữ các tài liệu trong các thư mục cụ thể (ví dụ: Function_Specs, Function_Designs, Test_Cases).

+ Quy trình phê duyệt tài liệu: Tài liệu trước khi được sử dụng phải được phê duyệt bởi trưởng nhóm hoặc người chịu trách nhiệm.

+ Theo dõi và cập nhật tài liệu: Khi có sự thay đổi trong yêu cầu, thiết kế, hoặc chiến lược kiểm thử, tài liệu liên quan cần được cập nhật và thông báo cho các thành viên trong nhóm.



Hình 4. Sơ đồ luồng công việc thủ tục kiểm soát tài liệu

- Vai trò của thành viên nhóm dự án:

+ Document manager (Quản lý tài liệu): Quản lý và lưu trữ tất cả các tài liệu liên quan đến chất lượng phần mềm, đảm bảo rằng tài liệu được cập nhật và kiểm soát phiên bản.

+ Document approver (Phê duyệt tài liệu): Trưởng nhóm hoặc người có thẩm quyền xem xét và phê duyệt tài liệu trước khi được sử dụng.

+ Team members (Các thành viên nhóm): Đảm bảo rằng tất cả các tài liệu có sự thay đổi hoặc cập nhật sẽ được thông báo và lưu trữ theo quy trình quản lý tài liệu.

4.6.4. Thủ tục kiểm soát thay đổi (Change control procedures)

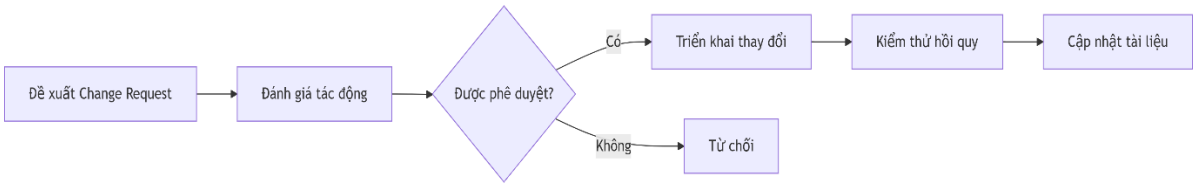
- Các bước thực hiện (Hình 5):

+ Đề xuất thay đổi: Các thay đổi phải được yêu cầu bởi người thay đổi yêu cầu (CR), trong đó mô tả rõ lý do thay đổi, tác động và phạm vi thay đổi.

+ Đánh giá thay đổi: Nhóm phát triển và các bên liên quan sẽ đánh giá tác động của thay đổi đối với hệ thống và tiến độ dự án.

+ Phê duyệt thay đổi: Chỉ những thay đổi được phê duyệt mới được triển khai vào mã nguồn và kiểm thử.

+ Theo dõi thay đổi: Sau khi thay đổi được triển khai, cần kiểm tra xem nó có gây ra lỗi hay ảnh hưởng đến các phần khác trong hệ thống không.



Hình 5. Sơ đồ luồng công việc thủ tục kiểm soát thay đổi

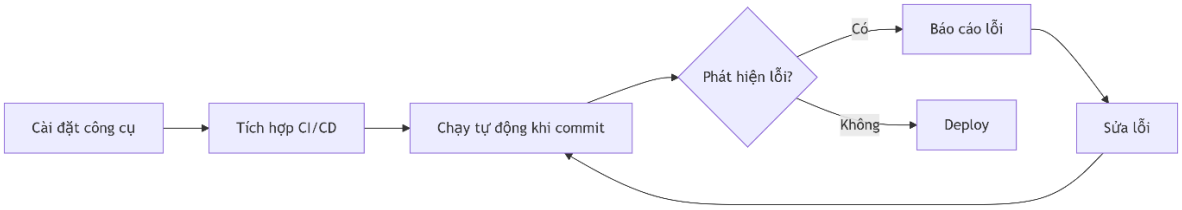
- Vai trò của thành viên nhóm dự án:
  + Change requester (CR-Người yêu cầu thay đổi): Đưa ra yêu cầu thay đổi, cung cấp thông tin chi tiết về lý do thay đổi và tác động của thay đổi.
  + Change evaluator (Người đánh giá thay đổi): Nhóm phát triển và các bên liên quan đánh giá tác động của thay đổi đối với hệ thống và tiến độ dự án.
  + Change approver (Người phê duyệt thay đổi): Trưởng nhóm hoặc người có thẩm quyền phê duyệt các thay đổi.
  + Change implementer (Người thực hiện thay đổi): Thực hiện thay đổi trong mã nguồn và đảm bảo rằng thay đổi được kiểm thử và tích hợp vào hệ thống.

4.6.5. Thủ tục kiểm thử mã nguồn tự động (Automated code testing procedures)

- Các bước thực hiện (Hình 6):
  + Cài đặt công cụ tự động hóa kiểm thử: Sử dụng các công cụ như ESLint (để kiểm tra cú pháp và quy tắc mã nguồn), Jest (để kiểm tra tính năng của mã nguồn).

+ Tích hợp với CI/CD pipeline: Đảm bảo rằng các bài kiểm thử tự động được tích hợp vào quy trình CI/CD (ví dụ: Jenkins, GitHub Actions) để chạy tự động mỗi khi có sự thay đổi trong mã nguồn.

+ Phân tích kết quả kiểm thử: Sau mỗi lần kiểm thử tự động, kết quả cần được phân tích và lỗi được báo cáo lại cho các lập trình viên để sửa chữa.



Hình 6. Sơ đồ luồng công việc thủ tục kiểm thử mã nguồn tự động

- Vai trò của thành viên nhóm dự án:

+ CI/CD engineer (Kỹ sư CI/CD): Thiết lập và duy trì quy trình tự động hóa kiểm thử trong pipeline CI/CD, tích hợp các công cụ kiểm thử tự động.

+ Developer (Lập trình viên): Viết và kiểm tra các unit test tự động cho mã nguồn của mình.

+ Automated test analyst (Chuyên viên kiểm thử tự động): Quản lý và phân tích kết quả kiểm thử tự động, báo cáo lỗi và yêu cầu sửa chữa.

4.6.6. Thủ tục báo cáo và phản hồi (Reporting and feedback procedures)

- Các bước thực hiện (Hình 7):

+ Báo cáo lỗi (Defect Report): Khi phát hiện lỗi trong quá trình kiểm thử hoặc review mã nguồn, lỗi cần được ghi lại chi tiết (mô tả lỗi, mức độ nghiêm trọng, tình trạng lỗi).

+ Báo cáo tổng kết kiểm thử (Test Summary Report): Sau mỗi vòng kiểm thử, kết quả kiểm thử phải được báo cáo, thống kê số lượng test case đã pass/fail và các lỗi đã phát hiện.

+ Báo cáo retrospective: Sau mỗi sprint, đội ngũ phát triển cần tổng kết các vấn đề đã làm tốt và chưa tốt trong quá trình phát triển phần mềm và lên kế hoạch cải tiến cho các sprint tiếp theo.



Hình 7. Sơ đồ luồng công việc thủ tục báo cáo và phản hồi

- Vai trò của thành viên nhóm dự án:
  + Test Lead (Trưởng nhóm kiểm thử): Chịu trách nhiệm tổng hợp và báo cáo kết quả kiểm thử, báo cáo các lỗi đã phát hiện.
  + Project Manager (Quản lý dự án): Nhận các báo cáo tổng kết kiểm thử, giúp theo dõi tiến độ và hỗ trợ đưa ra các quyết định cải tiến.
  + Team Members (Các thành viên nhóm): Cung cấp thông tin phản hồi về các vấn đề trong quá trình phát triển phần mềm và tham gia vào các cuộc họp tổng kết sau sprint.

2.2.4.4. Thiết lập quy trình QA

Thiết lập quy trình (hoặc thủ tục) trong đảm bảo chất lượng phần mềm (SQA) là việc xây dựng các hướng dẫn chi tiết, bước thực hiện cụ thể và quy tắc chuẩn hóa nhằm kiểm soát chất lượng sản phẩm trong suốt vòng đời phát triển phần mềm thông qua:

- Review (rà soát): Kiểm tra tĩnh (static testing) các tài liệu, mã nguồn, thiết kế.
- Kiểm thử (testing): Kiểm tra động (dynamic testing) để phát hiện lỗi khi chạy hệ thống.

Mục tiêu chính của thiết lập quy trình QA bao gồm:

- Đảm bảo tính nhất quán trong cách làm việc của team.
- Phòng ngừa rủi ro bằng cách tuân thủ các bước đã được kiểm chứng.
- Đo lường được hiệu quả chất lượng thông qua các tiêu chí rõ ràng.

Các quy trình QA cần xây dựng trong bất kỳ dự án phần mềm nào:

(1) Thủ tục kiểm tra mã nguồn:

+ Các mã nguồn phải tuân thủ các quy định về coding standards đã thiết lập (đã đề cập ở trên).

+ Mã nguồn cần phải trải qua quy trình review mã nguồn trước khi được tích hợp vào kho mã chính.

+ Mã cần phải được kiểm tra bằng các công cụ tự động hóa kiểm thử (như SonarQube) để phát hiện các lỗi tiềm ẩn (coding flaws, security vulnerabilities).

(2) Quy trình kiểm thử phần mềm:

+ Trước khi bắt đầu kiểm thử, cần phải có một ***Kế hoạch kiểm thử*** đầy đủ, chi tiết, xác định rõ phạm vi và mục tiêu kiểm thử.

+ Kiểm thử cần phải được thực hiện theo các giai đoạn đã định (unit test, integration test, system test).

+ Tất cả các trường hợp kiểm thử phải được ghi lại trong danh sách Test case, với kết quả kiểm thử chi tiết và báo cáo các lỗi phát hiện.

(3) Quy trình báo cáo lỗi:

+ Lỗi phát hiện trong quá trình kiểm thử cần được ghi lại đầy đủ trong Defect Report, bao gồm mức độ nghiêm trọng, mô tả lỗi, trạng thái xử lý và lịch trình xử lý.

+ Các lỗi quan trọng hoặc nghiêm trọng phải được giải quyết ngay lập tức, trong khi các lỗi ít nghiêm trọng có thể được xử lý trong các sprint sau.

(4) Quy trình kiểm tra hiệu suất:

+ Kiểm thử hiệu suất (performance testing) cần được thực hiện để đảm bảo phần mềm có thể đáp ứng các yêu cầu về hiệu suất đã được xác định trong tài liệu yêu cầu.

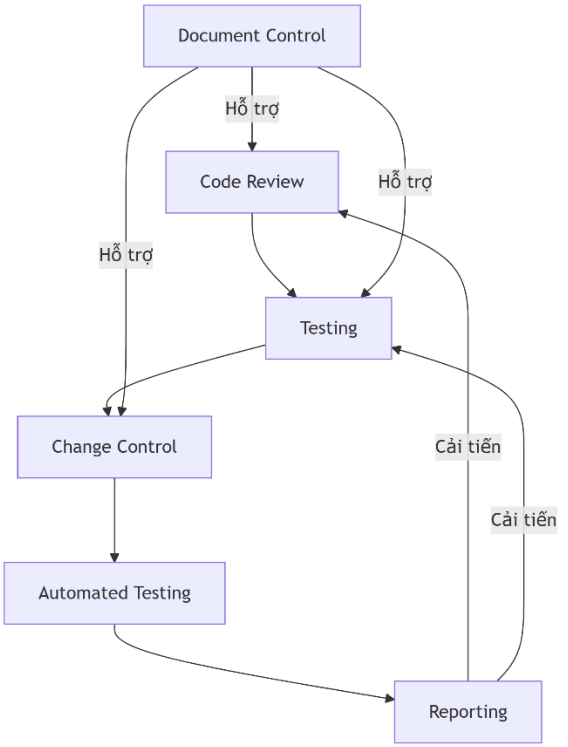
+ Các công cụ kiểm thử hiệu suất như JMeter có thể được sử dụng để kiểm tra khả năng chịu tải và các vấn đề tiềm ẩn trong khi sử dụng phần mềm với lượng người dùng lớn.

(5) Quy trình bảo trì chất lượng phần mềm:

+ Sau khi phần mềm đã được phát triển và triển khai, vẫn cần có các thủ tục để duy trì chất lượng phần mềm thông qua các quy trình kiểm thử định kỳ và cập nhật.

+ Tất cả các lỗi được phát hiện sau khi triển khai cần được ghi nhận và xử lý kịp thời.

Mối quan hệ giữa các quy trình trên Hình 2.



Hình 2. Mối quan hệ giữa các quy trình trong QA

2.2.4.5. Thiết lập danh mục kiểm tra

Danh mục kiểm tra (checklist) là công cụ chuẩn hóa và hệ thống hóa các tiêu chí đánh giá chất lượng, giúp đội ngũ phát triển và SQA tránh bỏ sót lỗi quan trọng. Checklist giúp:

- Giảm sai sót: Tránh bỏ qua lỗi nghiêm trọng do thiếu hệ thống.

- Tiết kiệm thời gian: Không cần "nghĩ lại từ đầu" mỗi lần test/review.

- Nhất quán chất lượng: Áp dụng cùng tiêu chuẩn cho mọi thành viên.

- Đo lường được: Dễ dàng đánh giá tỉ lệ hoàn thành công việc.

Test checklist, code review và Checklist cho tài liệu là các checklist quan trọng trong QA:

- Test checklist: Đảm bảo kiểm thử bao phủ đầy đủ các yêu cầu, chức năng và kịch bản quan trọng.

- Code review: Đảm bảo mã nguồn đáp ứng tiêu chuẩn kỹ thuật, dễ bảo trì và ít lỗi tiềm ẩn.

- Checklist cho tài liệu: dùng cho reviewer chính, để rà soát lại tài liệu xem có đủ các thành phần không? Có bị thiếu các mô tả không.

5. CÔNG CỤ HỖ TRỢ

Theo dõi chất lượng mã nguồn: SonarQube (kiểm tra chất lượng code: lỗi coding style, bugs tiềm ẩn,...), CodeClimate,…

Quản lý tài liệu và cải tiến quy trình: Confluence (dùng ghi nhận Retrospective và kế hoạch cải tiến), Notion,…

Thu thập phản hồi người dùng: Google Forms (tạo khảo sát nhanh với người dùng sau khi UAT), UserVoice,…

Quản lý kiểm thử: TestRail, Zephyr;

Theo rõi lỗi: Jira, Bugzilla;

Tự động hóa kiểm thử: Selenium, Postman